

Early Neural Language Model 学习笔记

这份笔记记录了我们从“词是怎么变成向量的”一路讨论到 early neural language model 的全过程。重点不只是公式，而是理解：为什么 NLP 会从 one-hot 时代进入 embedding 时代，以及 embedding 到底革命在哪里。

1. 模型到底在干什么

整个模型的目标其实非常朴素：给定前面的词（context），预测下一个词。例如：I love deep ____
模型会输出：P(learning | context) P(music | context) P(dogs | context) 最后选概率最大的那个。这其实就是 language model。

2. 第一步：token -> index

神经网络不能直接处理字符串。所以：deep -> 731 learning -> 912 这一步本质只是“编号”。通常做法：1. 扫描整个语料库 2. 建立 vocabulary 3. 按词频排序 4. 给每个词一个整数 ID 例如：the -> 0 of -> 1 and -> 2 注意：index 本身没有任何语义。5231 不比 87 “更高级”。它只是矩阵索引。

3. 第二步：embedding lookup

模型内部维护一个巨大矩阵： $C \in \mathbb{R}^{|V| \times d}$ 其中： $|V|$ = 词表大小 d = embedding 维度 例如： $C[731]$ = deep 的向量 $C[912]$ = learning 的向量 可能是：deep -> [0.2, -1.1] learning -> [0.7, 0.3]
这里不是“计算”，而是“查表”。所以叫：table lookup。

4. 最关键的问题：C 是写死的吗？

不是。这恰恰是 early neural language model 最革命的地方。embedding matrix C 也是训练参数。训练开始时： $C[\text{deep}] = [0.01, -0.03]$ 可能完全随机。随着模型不断预测下一个词：deep learning -> model deep learning -> network deep learning -> system 梯度会反向传播到 embedding matrix。于是：deep 的向量会被不断调整。

learning 的向量也会被不断调整。最终：语义相近的词会在向量空间中靠近。

5. 参数共享到底是什么意思

这里有个特别容易误解的地方。deep 和 cat 对应的 embedding 并不相同。即： $C[\text{deep}] \neq C[\text{cat}]$

所以不同词确实是不同参数。真正共享的是：同一个词无论出现在哪，都使用同一个 embedding。例如：I love deep learning deep models are powerful 两个 deep 都会查： $C[\text{deep}]$ 而不是：句首一个参数 句尾一个参数 这就是参数共享。

6. concat：把多个词向量拼起来

假设：deep $\rightarrow [1,2]$ learning $\rightarrow [3,4]$ 则： $x = [1,2,3,4]$ 这一步之后： x 不再是离散 token。而是上下文的连续表示。

7. Hidden Layer

之后进入 MLP： $h = \tanh(Wx + b)$ 如果有多层： $h_2 = \tanh(W_2h_1 + b_2)$ 这里本质是在不断混合上下文语义。为什么需要 $\tanh$ ？因为如果没有非线性： $W_2(W_1x)$ 仍然只是线性变换。多层就失去意义。 $\tanh$ 让模型能够表达复杂组合关系。

8. 输出层

最后： $z = Uh + c$ 这里： h 是 hidden state z 是 logits z 的维度等于词表大小。例如词表 50000： $z \in \mathbb{R}^{\{50000\}}$ 每一维：都代表“下一个词是该词”的打分。例如： $z[\text{cat}] = 2.1$ $z[\text{dog}] = 0.3$ $z[\text{banana}] = -1.7$

9. softmax

softmax 会把 logits 变成概率： $p_i = e^{z_i} / \sum e^{z_j}$ 于是：cat -> 0.72 dog -> 0.19 banana -> 0.0003 最终得到： $P(w_t | \text{context})$ 即：词表中每个词作为下一个词的概率。

10. U 和 C 的关系

embedding matrix C：词 -> 语义向量 输出矩阵 U：语义向量 -> 词概率 你甚至可以理解：U 的每一行都是某个词的“检测器”。例如：U[cat] 会学习：什么样的 hidden state 看起来像 cat。

11. 这套系统真正伟大的地方

在 one-hot 时代：cat = [0,0,1,0,0] dog = [0,1,0,0,0] 所有词完全独立。模型无法理解：cat 和 dog 很像。而 embedding 出现后：语义会自动形成几何结构。于是出现：king - man + woman $\approx$ queen 这种著名现象。

12. 为什么后来会有 BERT/GPT

early embedding 有一个巨大问题：一个词只有一个固定向量。例如：bank 既可能是：银行 也可能是：河岸 但 early embedding：bank 永远只有一个向量。所以后来：ELMo BERT GPT 开始使用 contextual embedding： $h_i = f(\text{context})$ 即：同一个词在不同上下文中，会生成不同表示。

13. 最本质的一句话

Early neural language model 的革命：把离散符号世界 映射到可学习的连续空间。即：token -> embedding vector -> gradient flow 于是：梯度第一次能够穿过“词”。这也是后面：word2vec Transformer GPT 一切现代 NLP 的祖先思想。
